Compte rendu — TP Android : Gestion des contacts V3

Étudiant	Obaid Allah — ISET Djerba
Sujet	Fragments · RecyclerView · Filtrage · Google Maps
Technologies	Kotlin, XML, View Binding, AndroidX Fragment / RecyclerView / Lifecycle, SDK Google Maps
Dépôt	Voir le lien GitHub déposé dans Google Classroom

1. Architecture des fragments

Une **seule activité**, `MainActivity`, contient un `FragmentContainerView` (`@id/fragment_container`) et rien d'autre. Les trois écrans sont des fragments qui se remplacent dynamiquement :

```
MainActivity (FragmentContainerView)
   └─ ListeContactsFragment ──clic──▶ DetailContactFragment ──« Voir sur la carte »──▶ CarteContactFragment
        (RecyclerView + recherche)      (fiche, édition, actions)                          (Google Maps)
```

Fragment	Rôle
`ListeContactsFragment`	Liste (RecyclerView), champ de recherche, compteur, message « aucun résultat »
`DetailContactFragment`	Consultation et modification, Annuler / Enregistrer, Appeler, E-mail, Galerie, Caméra, latitude / longitude
`CarteContactFragment`	Carte Google Maps centrée sur le contact, marqueur à son nom, message si coordonnées absentes ou invalides

Navigation. Chaque changement d'écran est une `FragmentTransaction` : `replace(R.id.fragment_container, …)` puis `addToBackStack(null)` (sauf pour le premier fragment). Le bouton Retour du système (ou le bouton ← de la carte) appelle `popBackStack()` et revient à l'écran précédent.

Rotation sans doublons. `MainActivity` n'ajoute `ListeContactsFragment` que si `savedInstanceState == null`. Après une rotation, le `FragmentManager` restaure lui-même les fragments et la pile de retour : en ajouter un autre créerait des doublons.

2. Choix de communication entre fragments

- **Liste → Détail et Détail → Carte** : un `Bundle` (`bundleOf`) créé dans `newInstance(...)`. On ne transmet que l'**identifiant** du contact (et, pour la carte, les coordonnées telles que saisies).
- **Détail → Liste (actualisation après enregistrement)** : un **`ContactsViewModel` partagé** (`by activityViewModels()`).

Justification du ViewModel partagé plutôt que la Fragment Result API.

Critère	ViewModel partagé (choisi)	Fragment Result API
Source de vérité	Une seule : la collection de contacts vit dans le ViewModel	Le résultat est un message ponctuel : il faut tout de même une source de données ailleurs
Mise à jour de la liste	Automatique : la liste *observe* `filtered` (LiveData)	Il faut écouter le résultat et recharger à la main
Rotation	Le ViewModel survit : données et texte du filtre conservés	Le message peut être perdu si le récepteur n'est pas actif
Filtre	Le texte de recherche est aussi dans le ViewModel, donc conservé en revenant du détail	À gérer séparément
Adapté à…	Des données partagées et durables entre plusieurs écrans	Un résultat unique renvoyé par un écran (ex. une boîte de dialogue)

3. RecyclerView

La `ListView` de la V2 est remplacée par une `RecyclerView` (`LinearLayoutManager`).

- **`ViewHolder`** (`ContactAdapter.ViewHolder`) conserve les vues d'une ligne (`ItemContactBinding`) : on évite de les rechercher à chaque défilement.
- **`ContactAdapter`** étend `ListAdapter<Contact, ViewHolder>` : `onCreateViewHolder` gonfle `item_contact.xml`, `onBindViewHolder` **remplit la ligne à partir du modèle** (photo, « prénom nom », téléphone, icône de position) — aucune donnée n'est figée dans le XML.
- **`DiffUtil.ItemCallback`** compare deux listes (`areItemsTheSame` sur l'`id`, `areContentsTheSame` sur l'égalité de la `data class`) : seules les lignes modifiées sont redessinées.
- Le clic transmet le **`Contact`** (donc son `id`), jamais la position de la ligne.

4. Stratégie de filtrage

- Le champ de recherche est au-dessus de la liste ; `doAfterTextChanged` envoie le texte au ViewModel à **chaque caractère** : pas de bouton de validation.
- `ContactFilter.filter(contacts, query)` est une **fonction pure** : elle reçoit la liste *complète* et le texte, et renvoie une *nouvelle* liste. Elle ne modifie jamais la liste d'origine ; effacer le texte renvoie donc tous les contacts.
- Critères : nom, prénom, « prénom nom » et « nom prénom », téléphone (espaces et séparateurs ignorés).
- Insensible à la **casse** et aux **accents** (`Normalizer` NFD) : « ines » trouve « Inès ».
- `ContactsViewModel.filtered` est un `MediatorLiveData` qui recalcule la liste quand les contacts **ou** le texte changent.
- Aucun résultat : un message « Aucun contact ne correspond à « … » » remplace la liste.

Éviter l'erreur d'indice. La position d'une ligne dans une liste filtrée ne correspond pas à celle du contact dans la liste complète. On ne l'utilise donc jamais : le clic envoie l'`id`, et le détail retrouve le contact avec `viewModel.getContact(id)`.

5. Coordonnées GPS et Google Maps

Modèle. `Contact` possède `latitude: Double?` et `longitude: Double?` (`null` = position non définie). Les contacts de démonstration ont tous des coordonnées valides, ou aucune.

Saisie et validation (`Coordinates.parse`) :

Saisie	Résultat
Deux champs vides	Pas de position (valide)
Un seul champ rempli	Erreur sur le champ vide
Texte non numérique (`abc`, `NaN`, `Infinity`)	Erreur « valeur numérique attendue »
Latitude hors −90…90, longitude hors −180…180	Erreur « hors limites »
`36,8065` (virgule)	Accepté

Les champs utilisent volontairement `inputType="text"` pour qu'on puisse saisir ou coller une valeur invalide et tester la validation.

Carte. `CarteContactFragment` contient un `SupportMapFragment` (dans un `FragmentContainerView`). Dans `onMapReady` : `moveCamera(newLatLngZoom(position, 14f))` et `addMarker(...title(nom complet))`. Si les coordonnées sont absentes ou invalides, la carte est masquée et un **message explicite** s'affiche à la place. Aucune autorisation de localisation : on affiche seulement des coordonnées déjà enregistrées.

Retour. La carte est ajoutée avec `addToBackStack` : Retour ramène à la fiche, avec les champs saisis conservés (l'état des vues est sauvegardé).

Clé API protégée. `app/build.gradle.kts` lit `MAPS_API_KEY` dans `local.properties` (ou dans la variable d'environnement du même nom) et la place dans `manifestPlaceholders` ; le manifeste contient seulement `android:value="${MAPS_API_KEY}"`. `local.properties` est dans `.gitignore` ; `local.properties.example` fournit un modèle **sans clé**. Il est aussi recommandé de restreindre la clé (paquet + SHA-1) dans Google Cloud.

6. Fonctionnalités de la V2 conservées

Fonctionnalité	Réalisation

Modèle Contact	`data class Contact` (nom, prénom, téléphone, e-mail, photo, + GPS)
Consultation / fiche détaillée	`DetailContactFragment`
Modification et enregistrement	`ContactsViewModel.update(...)` → liste actualisée
Appeler	`Intent.ACTION_DIAL` (`tel:`), numéro lu dans le champ
E-mail	`Intent.ACTION_SENDTO` (`mailto:`) + `EXTRA_EMAIL`
Galerie	`ActivityResultContracts.GetContent("image/*")`
Caméra	`ActivityResultContracts.TakePicture` + `FileProvider` (pas de permission CAMERA)
Annuler	Retour sans écrire dans le ViewModel : anciennes valeurs conservées
Champs obligatoires	Prénom, nom, téléphone (+ e-mail et téléphone validés)
Annulations / applications absentes	`ActivityNotFoundException` interceptée, résultat `null` / `false` géré, messages clairs

Les photos choisies sont copiées, **réduites à 800 px** et redressées selon l'EXIF (`PhotoStore`) dans le stockage privé de l'application.

7. Tests

7.1 Tests unitaires automatiques (`app/src/test`)

26 tests JUnit sur la logique pure (sans Android) : `ContactFilterTest` (11), `CoordinatesTest` (11), `ModelTest` (4) — casse, accents, téléphone, aucun résultat, effacement du filtre, identifiants, coordonnées valides / absentes / non numériques / hors limites, e-mail, téléphone.

Commande : `./gradlew test` — résultat : ☐ 26 / 26 réussis *(à confirmer sur votre machine)*.

7.2 Tests manuels (à exécuter sur émulateur ou téléphone)

Test	Résultat attendu	Observé
Filtrer par nom (`ben`)	Seul Ahmed Ben Ali reste	☐
Filtrer par prénom, casse différente (`SARRA`)	Sarra Trabelsi	☐
Filtrer par téléphone (`21 333` ou `21333`)	Sarra Trabelsi	☐
Recherche sans résultat (`zzz`)	Message « Aucun contact ne correspond… »	☐
Effacer le filtre (bouton ×)	Les 8 contacts réapparaissent	☐
Ouvrir un contact depuis une liste filtrée	La fiche affichée est celle du contact cliqué	☐
Modifier nom, téléphone, e-mail puis Enregistrer	Liste actualisée immédiatement	☐
Modifier la photo (galerie / caméra) puis Enregistrer	Nouvelle photo dans la fiche et dans la liste	☐
Modifier les coordonnées puis Enregistrer	Valeurs conservées en rouvrant la fiche	☐
Modifier puis Annuler	Anciennes valeurs conservées	☐
Champs obligatoires vides / e-mail invalide	Messages d'erreur, rien n'est enregistré	☐
Appeler	Composeur ouvert avec le numéro	☐
E-mail	Messagerie ouverte avec l'adresse préremplie	☐

Galerie / Caméra annulées	Message, aucune modification	☐
Appareil sans application (caméra, messagerie…)	Message, pas de plantage	☐
Carte d'un contact géolocalisé	Carte centrée, marqueur au nom du contact	☐
Coordonnées absentes (Inès Jaziri)	Message explicite à la place de la carte	☐
Coordonnées non numériques (`abc`) / hors limites (`95`, `200`)	Erreur à l'enregistrement et message sur la carte	☐
Retour depuis la carte	Retour sur la fiche, champs conservés	☐
Rotation (liste, fiche, carte)	Pas de fragment en double, filtre et saisie conservés	☐

7.3 Difficultés rencontrées *(à compléter avec votre expérience)*

Points sensibles du projet : empêcher les fragments en double après rotation (`savedInstanceState == null`) ; ne pas dépendre de la position après filtrage (identifiant) ; restaurer la saisie au retour de la carte ; photos volumineuses de la caméra (réduction + EXIF) ; configurer la clé Maps sans la publier.

8. Réponses aux questions de compréhension

1. Quelle différence existe-t-il entre une Activity et un Fragment ? Une `Activity` est un écran géré par le système (déclarée dans le manifeste, avec son propre cycle de vie et sa fenêtre). Un `Fragment` est une portion d'interface et de logique **réutilisable**, qui vit *dans* une activité et dépend de son cycle de vie : on peut en afficher plusieurs, les remplacer dynamiquement et les empiler dans une pile de retour, sans créer d'activité par écran.

2. Pourquoi utiliser `FragmentContainerView` et `addToBackStack()` ? `FragmentContainerView` est le conteneur recommandé pour héberger des fragments (il corrige des problèmes d'ordre de transitions et de z-order de `<fragment>` / `FrameLayout`). `addToBackStack()` enregistre la transaction : le bouton Retour l'annule et ramène au fragment précédent (liste → détail → carte → détail → liste) au lieu de quitter l'application.

3. Quel est le rôle de `RecyclerView.Adapter` et de `ViewHolder` ? L'adapter fait le lien entre les données (`Contact`) et les vues : il crée les lignes (`onCreateViewHolder`), les remplit (`onBindViewHolder`) et indique leur nombre. Le `ViewHolder` mémorise les vues d'une ligne pour éviter des recherches coûteuses ; la `RecyclerView` **recycle** les lignes qui sortent de l'écran, ce qui la rend rapide même pour de longues listes.

4. Comment éviter une erreur d'indice après filtrage ? En n'utilisant jamais la position de la ligne pour retrouver le contact. L'adapter affiche la liste filtrée (indices 0…n-1 de *cette* liste) mais le clic renvoie l'objet `Contact` ; on ouvre le détail avec son `id` et on le retrouve avec `getContact(id)`. De plus, le filtre produit une nouvelle liste sans toucher à la liste d'origine.

5. Pourquoi transmettre l'identifiant d'un contact plutôt que la position de sa ligne ? La position change dès que la liste change (filtre, tri, modification, suppression) et diffère entre liste filtrée et liste complète : elle pourrait désigner un autre contact. L'identifiant est stable et unique, et un `Bundle` de quelques octets suffit (pas besoin de sérialiser tout le contact).

6. Comment garantir qu'une clé Google Maps ne soit pas exposée dans GitHub ? La clé est mise dans `local.properties` (listé dans `.gitignore`) ou dans une variable d'environnement, puis injectée dans le manifeste à la compilation par `manifestPlaceholders` ; le dépôt ne contient qu'un modèle sans clé (`local.properties.example`). On vérifie avec `git status` avant chaque commit, on restreint la clé dans Google Cloud (nom de paquet + SHA-1), et si elle a été commitée par erreur on la **révoque** : la supprimer d'un commit ultérieur ne l'efface pas de l'historique.